

JavaScript 中的回调函数看这篇就够了

 疯狂的技术宅 2021-01-12 阅读 5 分钟

回调函数是每个前端程序员都应该知道的概念之一。回调可用于数组、计时器函数、`promise`、事件处理中。

本文将解释回调函数的概念，同时帮你区分两种回调：同步和异步。

回调函数

首先写一个向人打招呼的函数。

只需要创建一个接受 `name` 参数的函数 `greet(name)`。这个函数应返回打招呼的消息：

```
function greet(name) {
  return `Hello, ${name}!`;
}

greet('Cristina'); // => 'Hello, Cristinal'
```

如果向很多人打招呼该怎么办？可以用特殊的数组方法 `array.map()` 可以实现：

```
const persons = ['Cristina', 'Ana'];

const messages = persons.map(greet);
messages; // => ['Hello, Cristinal', 'Hello, Ana!']
```

`persons.map(greet)` 获取 `persons` 数组的所有元素，并分别用每个元素作为调用参数来调用 `greet()` 函数：`greet('Cristina')`、`greet('Ana')`。

有意思的是 `persons.map(greet)` 方法可以接受 `greet()` 函数作为参数。这样 `greet()` 就成了**回调函数**。

`persons.map(greet)` 是用另一个函数作为参数的函数，因此被称为**高阶函数**。

*回调函数*作为**高阶函数**的参数，高阶函数通过调用回调函数来执行操作。

重要的是高阶函数负责调用回调，并为其提供正确的参数。

在前面的例子中，高阶函数 `persons.map(greet)` 负责调用 `greet()` 函数，并分别把数组中所有的元素 `'Cristina'` 和 `Ana` 作为参数。

这就为识别回调提供了一条简单的规则。如果你定义了一个函数，并将其作参数提供给另一个函数的话，那么这就创建了一个回调。

你可以自己编写使用回调的高阶函数。下面是 `array.map()` 方法的等效版本：

```
function map(array, callback) {
  const mappedArray = [];
  for (const item of array) {
    mappedArray.push(
      callback(item)
    );
  }
  return mappedArray;
}

function greet(name) {
  return `Hello, ${name}!`;
}

const persons = ['Cristina', 'Ana'];

const messages = map(persons, greet);messages; // => ['Hello, Cristinal', 'Hello, Ana!']
```

`map(array, callback)` 是一个高阶函数，因为它用回调函数作为参数，然后在其主体内部调用该回调函数：`callback(item)`。

注意，常规函数（用关键字 `function` 定义）或箭头函数（用粗箭头 `=>` 定义）同样可以作为回调使用。

同步回调

回调的调用方式有两种：**同步**和**异步**回调。

同步回调是“阻塞”的：高阶函数直到回调函数完成后才继续执行。

例如，调用 `map()` 和 `greet()` 函数。

```
function map(array, callback) {
  console.log('map() starts');
  const mappedArray = [];
  for (const item of array) { mappedArray.push(callback(item)) }
  console.log('map() completed');
  return mappedArray;
}

function greet(name) {
  console.log('greet() called');
  return `Hello, ${name}!`;
}

const persons = ['Cristina'];

map(persons, greet);
// logs 'map() starts'
// logs 'greet() called'
// logs 'map() completed'
```

其中 `greet()` 是同步回调。

同步回调的步骤：

1. 高阶函数开始执行：`'map() starts'`
2. 回调函数执行：`'greet() called'`
3. 最后高阶函数完成它自己的执行过程：`'map() completed'`

同步回调的例子

许多原生 `JavaScript` 类型的方法都使用同步回调。

最常用的是 `array` 的方法，例如：`array.map(callback)`、`array.forEach(callback)`、`array.find(callback)`、`array.filter(callback)`、`array.reduce(callback, init)`

```
// Examples of synchronous callbacks on arrays
const persons = ['Ana', 'Elena'];

persons.forEach(
  function callback(name) { console.log(name); }
);
// logs 'Ana'
// logs 'Elena'

const nameStartingA = persons.find(
  function callback(name) { return name[0].toLowerCase() === 'a'; }
);
nameStartingA; // => 'Ana'

const countStartingA = persons.reduce(
  function callback(count, name) { const startsA = name[0].toLowerCase() === 'a';
    return startsA ? count + 1 : count; },
  0
);
countStartingA; // => 1
```

字符串类型的 `string.replace(callback)` 方法也能接受同步执行的回调。

```
// Examples of synchronous callbacks on strings
const person = 'Cristina';

// Replace 'i' with '1'
person.replace(/./g,
  function(char) { return char.toLowerCase() === 'i' ? '1' : char; }
); // => 'Cr1st1na'
```

异步回调

异步回调是“非阻塞的”：高阶函数无需等待回调完成即可完成其执行。高阶函数可确保稍后在特定事件上执行回调。

在以下的例子中，`later()` 函数的执行延迟了 2 秒：

```
console.log('setTimeout() starts');
setTimeout(function later() {
  console.log('later() called');
}, 2000);
console.log('setTimeout() completed');
```

```
// logs 'setTimeout() starts'
// logs 'setTimeout() completed'
// logs 'later() called' (after 2 seconds)
```

`later()` 是一个异步回调，因为 `setTimeout(later, 2000)` 启动并完成了执行，但是 `later()` 在 2 秒后执行。

异步调用回调的步骤：

1. 高阶函数开始执行：`'setTimeout()starts'`
2. 高阶函数完成其执行：`'setTimeout() completed'`
3. 回调函数在 2 秒钟后执行：`'later() called'`

异步回调的例子

计时器函数异步调用回调：

```
setTimeout(function later() {
  console.log('2 seconds have passed!');
}, 2000);
// After 2 seconds logs '2 seconds have passed!'
```

```
setInterval(function repeat() {
  console.log('Every 2 seconds');
}, 2000);
// Each 2 seconds logs 'Every 2 seconds!'
```

DOM 事件侦听器还异步调用事件处理函数（回调函数的子类型）：

```
const myButton = document.getElementById('myButton');

myButton.addEventListener('click', function handler() {
  console.log('Button clicked!');
});
// Logs 'Button clicked!' when the button is clicked
```

4. 异步回调函数与异步函数

在函数定义之前加上特殊关键字 `async` 会创建一个异步函数：

```
async function fetchUserNames() {
  const resp = await fetch('https://api.github.com/users?per_page=5');
  const users = await resp.json();
  const names = users.map(({ login }) => login);
  console.log(names);
}
```

`fetchUserNames()` 是异步的，因为它以 `async` 为前缀。函数 `await fetch('https://api.github.com/users?per_page=5')` 从 GitHub 上获取前 5 个用户。然后从响应对象中提取 JSON 数据：`await resp.json()`。

异步函数是 `promise` 之上的语法糖。当遇到表达式 `await <promise>`（调用 `fetch()` 会返回一个 `promise`）时，异步函数会暂停执行，直到 `promise` 被解决。

异步回调函数和异步函数是**不同**的两个术语。

异步回调函数由高阶函数以非阻塞方式执行。但是异步函数在等待 `promise` (`await <promise>`) 解析时会暂停执行。

但是你可以把异步函数用作异步回调！

让我们把异步函数 `fetch UserNames()` 设为异步回调，只需单击按钮即可调用：

```
const button = document.getElementById('fetchUsersButton');

button.addEventListener('click', fetchUserNames);
```

总结

回调是一个可以作为参数传给另一个函数（高阶函数）执行的函数。

回调函数有两种：同步和异步。

同步回调是**阻塞**的。

异步回调是**非阻塞**的。

最后考考你：`setTimeout(callback, 0)` 执行 `callback` 时是同步还是异步的？